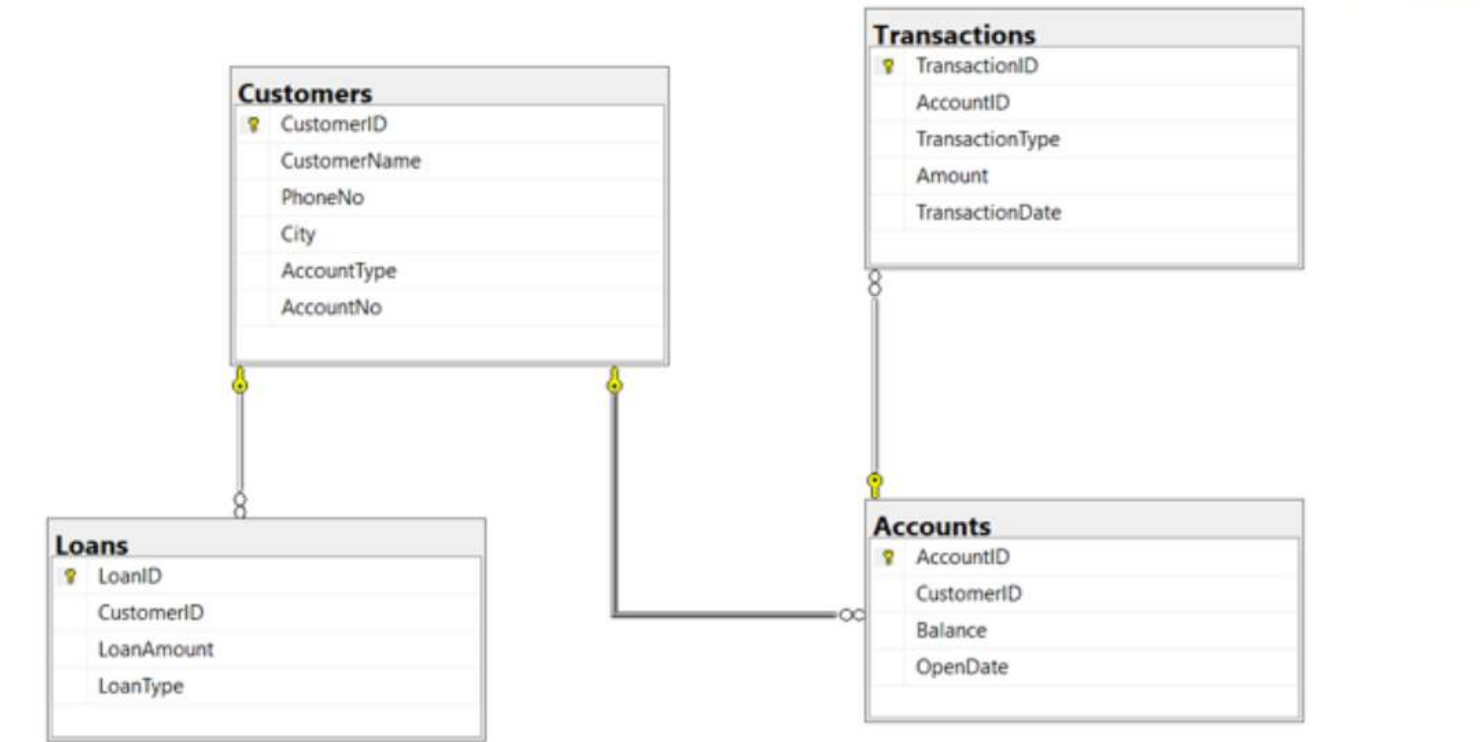


BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details



BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q1) Display customer names, account numbers, and account balances using INNER JOIN

```
SELECT c.CustomerName, a.AccountID, a.Balance
FROM Customers c
INNER JOIN Accounts a
ON c.CustomerID = a.CustomerID;
```

 Results  Messages

	CustomerName	AccountID	Balance
1	Rahul Sharma	1001	55000.00
2	Sneha Patil	1002	120000.00
3	Aman Verma	1003	35000.00
4	Priya Singh	1004	98000.00
5	Karan Mehta	1005	75000.00
6	Neha Joshi	1006	150000.00
7	Rohit Kumar	1007	42000.00
8	Pooja Sharma	1008	88000.00
9	Vivek Shah	1009	200000.00
10	Anjali Verma	1010	67000.00

INNER JOIN
combines rows from
Customers and
Accounts tables,
returning only the
records where a
matching
CustomerID exists in
both tables.

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q2) Find the top 3 customers with the highest account balances.

```
SELECT TOP 3 c.CustomerName, a.AccountID, a.Balance
FROM Customers c
INNER JOIN Accounts a
ON c.CustomerID = a.CustomerID
ORDER BY a.Balance DESC;
```

	CustomerName	AccountID	Balance
1	Vivek Shah	1009	200000.00
2	Neha Joshi	1006	150000.00
3	Sneha Patil	1002	120000.00

TOP is used to limit the number of rows returned by a query, showing only the first N records from the result set.

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q3) Show all customers who have taken loans along with loan amount and loan type.

```
SELECT c.CustomerName, l.LoanAmount, l.LoanType
FROM Customers c
INNER JOIN Loans l
ON c.CustomerID = l.CustomerID;
```

 Results  Messages

	CustomerName	LoanAmount	LoanType
1	Rahul Sharma	500000.00	Home Loan
2	Sneha Patil	200000.00	Car Loan
3	Priya Singh	100000.00	Education Loan
4	Karan Mehta	300000.00	Business Loan
5	Neha Joshi	150000.00	Personal Loan
6	Pooja Sharma	250000.00	Home Loan
7	Vivek Shah	400000.00	Business Loan
8	Anjali Verma	180000.00	Car Loan

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q4) Find the total deposited amount and total withdrawn amount separately.

```
SELECT TransactionType, SUM(Amount) as Total_Amount  
FROM Transactions  
GROUP BY TransactionType;
```

 Results		 Messages
	TransactionType	Total_Amount
1	Deposit	121000.00
2	Withdraw	35000.00

GROUP BY: Groups similar rows for aggregate calculations.

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q5) Display customer-wise total transaction amount using GROUP BY.

```
SELECT c.CustomerName, SUM(t.Amount) AS TotalTransactionAmount
FROM Customers c JOIN Accounts a
ON c.CustomerID = a.CustomerID JOIN Transactions t
ON a.AccountID = t.AccountID
GROUP BY c.CustomerName;
```

	CustomerName	TotalTransactionAmount
1	Aman Verma	8000.00
2	Anjali Verma	2000.00
3	Karan Mehta	15500.00
4	Neha Joshi	7000.00
5	Pooja Sharma	4500.00
6	Priya Singh	15000.00
7	Rahul Sharma	15000.00
8	Rohit Kumar	24000.00
9	Sneha Patil	35000.00
10	Vivek Shah	30000.00

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q6) Find customers whose balances are greater than the average bank balance.

```
SELECT c.customername , a.balance
FROM Customers c JOIN Accounts a
ON c.CustomerID = a.CustomerID
WHERE a.balance > (SELECT AVG(balance) FROM Accounts);
```

Results		Messages
	customername	balance
1	Sneha Patil	120000.00
2	Priya Singh	98000.00
3	Neha Joshi	150000.00
4	Vivek Shah	200000.00

WHERE: Filters rows based on a specified condition.

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q7) Show the highest transaction amount performed by each customer.

```
SELECT c.customername , MAX(t.amount) as MAX_ammount
FROM Customers c JOIN ACCOUNTS a
ON c.CustomerID = a.CustomerID JOIN Transactions t
ON a.AccountID = t.AccountID
GROUP BY c.CustomerName;
```

Results		Messages
	customername	MAX_ammount
1	Aman Verma	5000.00
2	Anjali Verma	2000.00
3	Karan Mehta	12000.00
4	Neha Joshi	7000.00
5	Pooja Sharma	4500.00
6	Priya Singh	15000.00
7	Rahul Sharma	10000.00
8	Rohit Kumar	15000.00
9	Sneha Patil	25000.00
10	Vivek Shah	30000.00

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q8) Display all customers who have not taken any loans using LEFT JOIN.

```
SELECT c.CustomerName
FROM Customers c LEFT JOIN Loans l
ON c.CustomerID = l.CustomerID
WHERE l.CustomerID IS NULL;
```

Results		Messages	
	CustomerName		
1	Aman Verma		
2	Rohit Kumar		

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q9) Find the total number of transactions performed by each customer.

```
SELECT c.customername , COUNT(t.transactionid) as Total_Transactions
FROM Customers c JOIN Accounts a
ON c.CustomerID = a.CustomerID JOIN Transactions t
ON a.AccountID = t.AccountID
GROUP BY c.customername;
```

	customername	Total_Transactions
1	Aman Verma	2
2	Anjali Verma	1
3	Karan Mehta	2
4	Neha Joshi	1
5	Pooja Sharma	1
6	Priya Singh	1
7	Rahul Sharma	2
8	Rohit Kumar	2
9	Sneha Patil	2
10	Vivek Shah	1

PRIMARY KEY: Uniquely identifies each record in a table.

FOREIGN KEY: Links a table to another table using a reference column.

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q 10) Rank customers based on their account balances using RANK() window function.

```
SELECT c.CustomerName, a.Balance,  
RANK() OVER (ORDER BY a.Balance DESC) AS Balance_Rank  
FROM Customers c  
JOIN Accounts a  
ON c.CustomerID = a.CustomerID;
```

Results		Messages	
	CustomerName	Balance	Balance_Rank
1	Vivek Shah	200000.00	1
2	Neha Joshi	150000.00	2
3	Sneha Patil	120000.00	3
4	Priya Singh	98000.00	4
5	Pooja Sharma	88000.00	5
6	Karan Mehta	75000.00	6
7	Anjali Verma	67000.00	7
8	Rahul Sharma	55000.00	8
9	Rohit Kumar	42000.00	9
10	Aman Verma	35000.00	10

RANK(): Assigns a rank to each row based on a specified order.

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q 11) Display dense ranking of customers according to balance using DENSE_RANK().

```
SELECT c.CustomerName, a.Balance,  
DENSE_RANK() OVER (ORDER BY a.Balance DESC) AS Dense_Rank  
FROM Customers c  
JOIN Accounts a  
ON c.CustomerID = a.CustomerID;
```

	CustomerName	Balance	Dense_Rank
1	Vivek Shah	200000.00	1
2	Neha Joshi	150000.00	2
3	Sneha Patil	120000.00	3
4	Priya Singh	98000.00	4
5	Pooja Sharma	88000.00	5
6	Karan Mehta	75000.00	6
7	Anjali Verma	67000.00	7
8	Rahul Sharma	55000.00	8
9	Rohit Kumar	42000.00	9
10	Aman Verma	35000.00	10

DENSE_RANK(): Assigns a rank to each row without skipping rank numbers for ties.

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q 12) Show previous transaction amount using LAG() function.

```
SELECT TransactionID, TransactionDate, Amount,  
LAG(Amount) OVER (ORDER BY TransactionDate) AS Previous_Amount  
FROM Transactions;
```

Results		Messages		
	TransactionID	TransactionDate	Amount	Previous_Amount
1	1	2026-06-01	10000.00	NULL
2	2	2026-06-02	5000.00	10000.00
3	3	2026-06-02	25000.00	5000.00
4	4	2026-06-03	3000.00	25000.00
5	5	2026-06-04	15000.00	3000.00
6	6	2026-06-05	12000.00	15000.00
7	7	2026-06-05	7000.00	12000.00
8	8	2026-06-06	9000.00	7000.00
9	9	2026-06-06	4500.00	9000.00
10	10	2026-06-07	30000.00	4500.00
11	11	2026-06-07	2000.00	30000.00
12	12	2026-06-08	10000.00	2000.00
13	13	2026-06-08	5000.00	10000.00
14	14	2026-06-09	3500.00	5000.00
15	15	2026-06-09	15000.00	3500.00

LAG(): Retrieves a value from the previous row in a result set.

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q 13) Show next transaction amount using LEAD() function.

```
SELECT TransactionID, TransactionDate, Amount,  
LEAD(Amount) OVER (ORDER BY TransactionDate) AS Next_Amount  
FROM Transactions;
```

	TransactionID	TransactionDate	Amount	Next_Amount
1	1	2026-06-01	10000.00	5000.00
2	2	2026-06-02	5000.00	25000.00
3	3	2026-06-02	25000.00	3000.00
4	4	2026-06-03	3000.00	15000.00
5	5	2026-06-04	15000.00	12000.00
6	6	2026-06-05	12000.00	7000.00
7	7	2026-06-05	7000.00	9000.00
8	8	2026-06-06	9000.00	4500.00
9	9	2026-06-06	4500.00	30000.00
10	10	2026-06-07	30000.00	2000.00
11	11	2026-06-07	2000.00	10000.00
12	12	2026-06-08	10000.00	5000.00
13	13	2026-06-08	5000.00	3500.00
14	14	2026-06-09	3500.00	15000.00
15	15	2026-06-09	15000.00	NULL

LEAD(): Retrieves a value from the next row in a result set.

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q 14) Calculate running total of transaction amounts using SUM() OVER().

```
SELECT TransactionID, TransactionDate, Amount,  
SUM(Amount) OVER (ORDER BY TransactionDate) AS Running_Total  
FROM Transactions;
```

Results		Messages		
	TransactionID	TransactionDate	Amount	Running_Total
1	1	2026-06-01	10000.00	10000.00
2	2	2026-06-02	5000.00	40000.00
3	3	2026-06-02	25000.00	40000.00
4	4	2026-06-03	3000.00	43000.00
5	5	2026-06-04	15000.00	58000.00
6	6	2026-06-05	12000.00	77000.00
7	7	2026-06-05	7000.00	77000.00
8	8	2026-06-06	9000.00	90500.00
9	9	2026-06-06	4500.00	90500.00
10	10	2026-06-07	30000.00	122500.00
11	11	2026-06-07	2000.00	122500.00
12	12	2026-06-08	10000.00	137500.00
13	13	2026-06-08	5000.00	137500.00
14	14	2026-06-09	3500.00	156000.00
15	15	2026-06-09	15000.00	156000.00

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q 15) Find the second highest account balance using subquery or window function.

```
SELECT MAX(Balance) AS Second_Highest_Balance
FROM Accounts
WHERE Balance < (
    SELECT MAX(Balance)
    FROM Accounts
);
```



Results



Messages

	Second_Highest_Balance
1	150000.00

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q 16) Find customers who performed more than 2 transactions.

```
SELECT c.CustomerName, COUNT(t.TransactionID) AS Total_Transactions
FROM Customers c
JOIN Accounts a
ON c.CustomerID = a.CustomerID
JOIN Transactions t
ON a.AccountID = t.AccountID
GROUP BY c.CustomerName
HAVING COUNT(t.TransactionID) > 2;
```



Results



Messages

CustomerName	Total_Transactions
--------------	--------------------

BANK CUSTOMER ACCOUNTS OVERVIEW

Retrieve customer names with their account details

Q 17) Display customer-wise minimum and maximum transaction amounts.

```
SELECT c.CustomerName, MIN(t.Amount) AS Min_Transaction, MAX(t.Amount) AS Max_Transaction
FROM Customers c JOIN Accounts a
ON c.CustomerID = a.CustomerID
JOIN Transactions t
ON a.AccountID = t.AccountID
GROUP BY c.CustomerName;
```

Results		Messages	
	CustomerName	Min_Transaction	Max_Transaction
1	Aman Verma	3000.00	5000.00
2	Anjali Verma	2000.00	2000.00
3	Karan Mehta	3500.00	12000.00
4	Neha Joshi	7000.00	7000.00
5	Pooja Sharma	4500.00	4500.00
6	Priya Singh	15000.00	15000.00
7	Rahul Sharma	5000.00	10000.00
8	Rohit Kumar	9000.00	15000.00
9	Sneha Patil	10000.00	25000.00
10	Vivek Shah	30000.00	30000.00